

C Programming String

In C programming, array of character are called strings. A string is terminated by null character `/0`. For example:

```
"c string tutorial"
```

Here, "c string tutorial" is a string. When, compiler encounters strings, it appends null character at the end of string.

c		s	t	r	i	n	g		t	u	t	o	r	i	a	l	\0
---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	----

Declaration of strings

Strings are declared in C in similar manner as arrays. Only difference is that, strings are of char type.

```
char s[5];
```

s[0]	s[1]	s[2]	s[3]	s[4]

Strings can also be declared using pointer.

```
char *p
```

Initialization of strings

In C, string can be initialized in different number of ways.

```
char c[]="abcd";
```

OR,

```
char c[5]="abcd";
```

OR,

```
char c[]={ 'a','b','c','d','\0' };
```

OR;

```
char c[5]={ 'a','b','c','d','\0' };
```

c[0]	c[1]	c[2]	c[3]	c[4]
a	b	c	d	\0

String can also be initialized using pointers

```
char *c="abcd";
```

Reading Strings from user.

Reading words from user.

```
char c[20];  
  
scanf("%s",c);
```

String variable *c* can only take a word. It is because when white space is encountered, the `scanf()` function terminates.

Write a C program to illustrate how to read string from terminal.

```
#include <stdio.h>  
int main(){  
    char name[20];  
    printf("Enter name: ");  
    scanf("%s",name);  
    printf("Your name is %s.",name);  
    return 0;  
}
```

Output

```
Enter name: Dennis Ritchie  
  
Your name is Dennis.
```

Here, program will ignore Ritchie because, `scanf()` function takes only string before the white space.

Reading a line of text

C program to read line of text manually.

```
#include <stdio.h>  
int main(){  
    char name[30],ch;  
    int i=0;  
    printf("Enter name: ");  
    while(ch!='\n') // terminates if user hit enter  
    {  
        ch=getchar();  
        name[i]=ch;  
        i++;  
    }  
    name[i]='\0'; // inserting null character at end  
    printf("Name: %s",name);  
    return 0;  
}
```

This process to take string is tedious. There are predefined functions `gets()` and `puts` in C language to read and display string respectively.

```
int main(){
    char name[30];
    printf("Enter name: ");
    gets(name);    //Function to read string from user.
    printf("Name: ");
    puts(name);    //Function to display string.
    return 0;
}
```

Both, the above program has same output below:

Output

```
Enter name: Tom Hanks
```

```
Name: Tom Hanks
```

Passing Strings to Functions

String can be passed to function in similar manner as arrays as, string is also an array. Learn more about [passing array to a function](#).

```
#include <stdio.h>
void Display(char ch[]);
int main(){
    char c[50];
    printf("Enter string: ");
    gets(c);
    Display(c);    // Passing string c to function.
    return 0;
}
void Display(char ch[]){
    printf("String Output: ");
    puts(ch);
}
```

Here, string `c` is passed from `main()` function to user-defined function `Display()`. In function declaration, `ch[]` is the formal argument.

String handling functions

You can perform different type of string operations manually like: finding length of string, concatenating(joining) two strings etc. But, for programmers ease, many library function are defined under header file `<string.h>` to handle these commonly used task in C programming.

String Manipulations In C Programming Using Library Functions

Strings are often needed to be manipulated by programmer according to the need of a problem. All string manipulation can be done manually by the programmer but, this makes programming complex and large. To solve this, the C supports a large number of string handling functions.

There are numerous functions defined in "string.h" header file. Few commonly used string handling functions are discussed below:

Function	Work of Function
<u>strlen()</u>	Calculates the length of string
<u>strcpy()</u>	Copies a string to another string
<u>strcat()</u>	Concatenates(joins) two strings
<u>strcmp()</u>	Compares two string
<u>strlwr()</u>	Converts string to lowercase
<u>strupr()</u>	Converts string to uppercase

Strings handling functions are defined under "string.h" header file, i.e, you have to include the code below to run string handling functions.

```
#include <string.h>
```

gets() and puts()

Functions gets() and puts() are two string functions to take string input from user and display string respectively as mentioned in previous chapter.

```
#include<stdio.h>
int main(){
    char name[30];
    printf("Enter name: ");
    gets(name);    //Function to read string from user.
    printf("Name: ");
    puts(name);    //Function to display string.
    return 0;
}
```

Though, gets() and puts() function handle string, both these functions are defined in "stdio.h" header file.

Program to find the Frequency of Characters

```
#include <stdio.h>

int main(){
    char c[1000],ch;
    int i,count=0;
    printf("Enter a string: ");
    gets(c);
    printf("Enter a character to find frequency: ");
    scanf("%c",&ch);
    for(i=0;c[i]!='\0';i++)
    {
        if(ch==c[i])
            count++;
    }
    printf("Frequency of %c = %d", ch, count);
    return 0;
}
```

Find Number of Vowels, Consonants, Digits and White Space Character

```
#include<stdio.h>

int main(){
    char line[150];
    int i,v,c,ch,d,s,o;
    o=v=c=ch=d=s=0;
    printf("Enter a line of string:\n");
    gets(line);
    for(i=0;line[i]!='\0';i++)
    {
```

```

        if(line[i]=='a' || line[i]=='e' || line[i]=='i' || line[i]=='o' || line[i]=='u' || line[i]=='A' ||
line[i]=='E' || line[i]=='I' || line[i]=='O' || line[i]=='U')

            v++;

        else if((line[i]>='a'&& line[i]<='z') || (line[i]>='A'&& line[i]<='Z'))

            c++;

        else if(line[i]>='0'&&c<='9')

            d++;

        else if (line[i]==' ')

            s++;

    }

    printf("Vowels: %d",v);

    printf("\nConsonants: %d",c);

    printf("\nDigits: %d",d);

    printf("\nWhite spaces: %d",s);

    return 0;

}

```

Calculated Length without Using strlen() Function

```

#include <stdio.h>

int main()

{

    char s[1000],i;

    printf("Enter a string: ");

    scanf("%s",s);

    for(i=0; s[i]!='\0'; i++);

    printf("Length of string: %d",i);

    return 0;}

```

String Concatenation

You can concatenate two strings easily using standard library function `strcat()` but, this program concatenates two strings manually without using `strcat()` function.

```
#include <stdio.h>

int main()
{
    char s1[100], s2[100], i, j;

    printf("Enter first string: ");
    scanf("%s",s1);

    printf("Enter second string: ");
    scanf("%s",s2);

    for(i=0; s1[i]!='\0'; i++); /* i contains length of string s1. */
    for(j=0; s2[j]!='\0'; j++, i++)
    {
        s1[i]=s2[j];
    }

    s1[i]='\0';

    printf("After concatenation: %s",s1);

    return 0;
}
```

Program to copy contents of one string to another string

You can use the strcpy() function to copy the content of one string to another but, this program copies the content of one string to another manually without using strcpy() function.

```
#include <stdio.h>

int main()
{
    char s1[100], s2[100], i;
    printf("Enter string s1: ");
    scanf("%s",s1);
    for(i=0; s1[i]!='\0'; i++)
    {
        s2[i]=s1[i];
    }
    s2[i]='\0';
    printf("String s2: %s",s2);
    return 0;
}
```

Program to check for String Palindrome

Palindrome is a number or string that remains the same even if reversed for example , MALAYALAM if reversed again it is MALAYALAM

```
/*
* C program to read a string and check if it's a palindrome, without
* using library functions. Display the result.
*/
```

```

#include <stdio.h>

#include <string.h>

void main()
{
    char string[25], reverse_string[25] = {"\0"};

    int i, length = 0, flag = 0;

    printf("Enter a string \n");

    gets(string);

    /* keep going through each character of the string till its end */
    for (i = 0; string[i] != '\0'; i++)
    {
        length++;
    }

    for (i = length - 1; i >= 0; i--)
    {
        reverse_string[length - i - 1] = string[i];
    }

    /*
    * Compare the input string and its reverse. If both are equal
    * then the input string is palindrome.
    */

    for (i = 0; i < length; i++)
    {
        if (reverse_string[i] == string[i])

            flag = 1;

        else

```

```

        flag = 0;
    }
    if (flag == 1)
        printf("%s is a palindrome \n", string);
    else
        printf("%s is not a palindrome \n", string);
}

```

Program to accept 2 strings and compare them

```

/*
* C Program to accepts two strings and compare them. Display
* the result whether both are equal, or first string is greater
* than the second or the first string is less than the second string
*/
#include <stdio.h>

void main()
{
    int count1 = 0, count2 = 0, flag = 0, i;
    char string1[10], string2[10];

    printf("Enter a string:");
    gets(string1);
    printf("Enter another string:");
    gets(string2);

    /* Count the number of characters in string1 */
    while (string1[count1] != '\0')
        count1++;

    /* Count the number of characters in string2 */

```

```
while (string2[count2] != '\0')
    count2++;
i = 0;
while ((i < count1) && (i < count2))
{
    if (string1[i] == string2[i])
    {
        i++;
        continue;
    }
    if (string1[i] < string2[i])
    {
        flag = -1;
        break;
    }
    if (string1[i] > string2[i])
    {
        flag = 1;
        break;
    } }
if (flag == 0)
    printf("Both strings are equal \n");
if (flag == 1)
    printf("String1 is greater than string2 \n", string1, string2);
if (flag == -1)
    printf("String1 is less than string2 \n", string1, string2);
}
```

Program to sort n names alphabetically

```
/*  
 * C program to read N names, store them in the form of an array  
 * and sort them in alphabetical order. Output the given names and  
 * the sorted names in two columns side by side.  
 */  
  
#include <stdio.h>  
  
#include <string.h>  
  
void main()  
{  
    char name[10][8], tname[10][8], temp[8];  
    int i, j, n;  
  
    printf("Enter the value of n \n");  
    scanf("%d", &n);  
    printf("Enter %d names\n", n);  
    for (i = 0; i < n; i++)  
    {  
        scanf("%s", name[i]);  
        strcpy(tname[i], name[i]);  
    }  
    for (i = 0; i < n - 1 ; i++)  
    {  
        for (j = i + 1; j < n; j++)  
        {
```

```

        if (strcmp(name[i], name[j]) > 0)
        {
            strcpy(temp, name[i]);
            strcpy(name[i], name[j]);
            strcpy(name[j], temp);
        }
    }

    printf("\n-----\n");
    printf("Input Names    Sorted names\n");
    printf("-----\n");
    for (i = 0; i < n; i++)
    {
        printf("%s\t\t%s\n", tname[i], name[i]);
    }
    printf("-----\n");
}

```